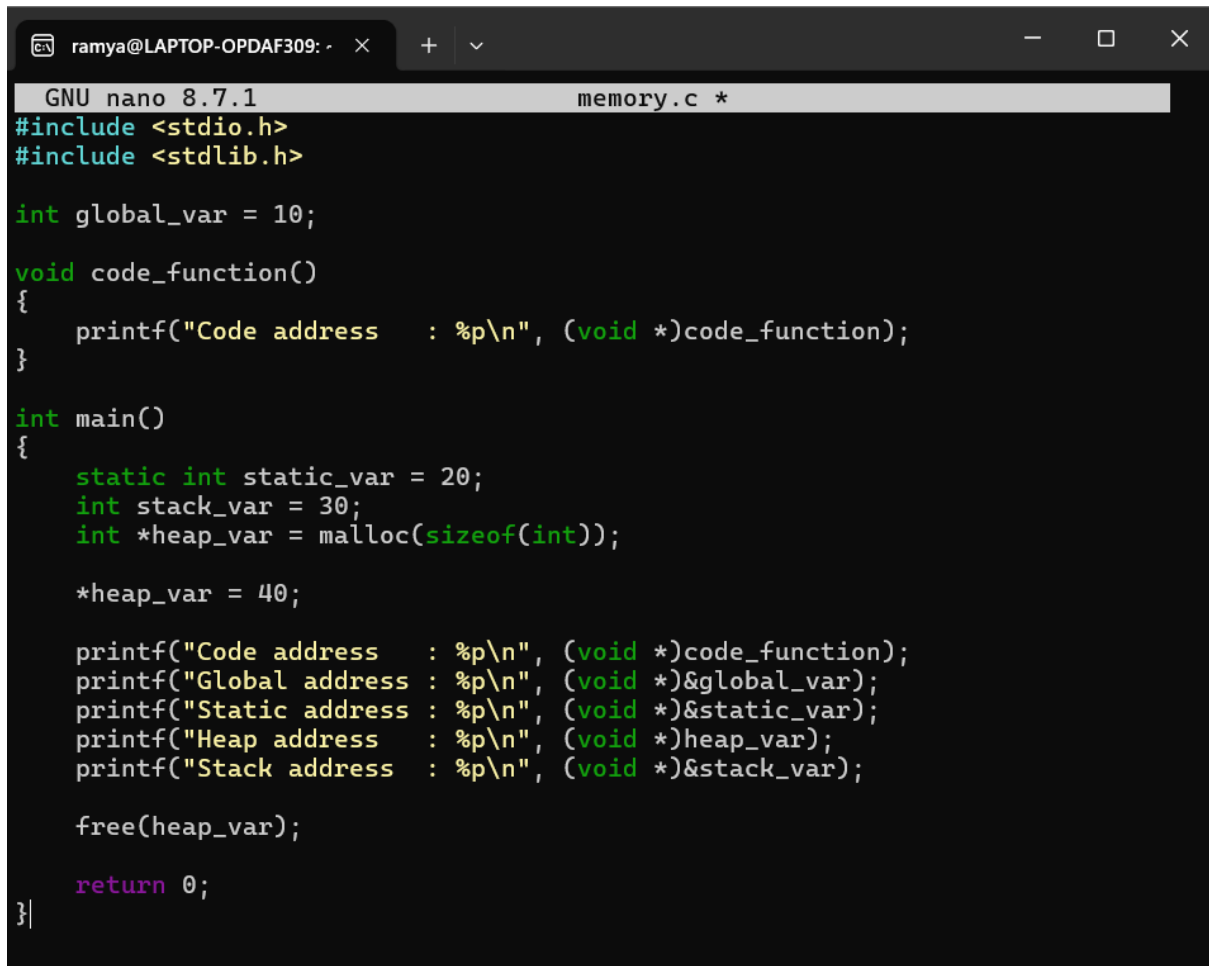


WEEK-7

Task 1 : Write a program that prints the addresses Of code, global, static, heap, and stack variables. Analyze the Linux process address space layout and explain each memory segment.

A screenshot of a terminal window with a dark background. The window title bar shows 'ramya@LAPTOP-OPDAF309: ~' and standard window controls. The terminal displays the GNU nano 8.7.1 editor editing a file named 'memory.c'. The code is a C program that prints the addresses of various memory segments. It includes `<stdio.h>` and `<stdlib.h>`. It defines a global variable `global_var` with value 10, a function `code_function` that prints its own address, and a `main` function. In `main`, it declares a static variable `static_var` with value 20, a stack variable `stack_var` with value 30, and a heap variable `heap_var` allocated with `malloc`. It then prints the addresses of `code_function`, `global_var`, `static_var`, `heap_var`, and `stack_var`. Finally, it frees `heap_var` and returns 0.

```
GNU nano 8.7.1 memory.c *
#include <stdio.h>
#include <stdlib.h>

int global_var = 10;

void code_function()
{
    printf("Code address    : %p\n", (void *)code_function);
}

int main()
{
    static int static_var = 20;
    int stack_var = 30;
    int *heap_var = malloc(sizeof(int));

    *heap_var = 40;

    printf("Code address    : %p\n", (void *)code_function);
    printf("Global address : %p\n", (void *)&global_var);
    printf("Static address : %p\n", (void *)&static_var);
    printf("Heap address  : %p\n", (void *)heap_var);
    printf("Stack address : %p\n", (void *)&stack_var);

    free(heap_var);

    return 0;
}
```

ramya@LAPTOP-OPDAF309: ~

+ v

```
ramya@LAPTOP-OPDAF309:~$ mkdir prac7
```

```
ramya@LAPTOP-OPDAF309:~$ cd prac7
```

```
ramya@LAPTOP-OPDAF309:~/prac7$ nano memory.c
```

```
ramya@LAPTOP-OPDAF309:~/prac7$ gcc memory.c -o memory
```

```
ramya@LAPTOP-OPDAF309:~/prac7$ ./memory
```

```
Code address      : 0x62d26e9c81a9
```

```
Global address    : 0x62d26e9cb010
```

```
Static address    : 0x62d26e9cb014
```

```
Heap address      : 0x62d2a7486010
```

```
Stack address     : 0x7ffc8c334a6c
```

```
ramya@LAPTOP-OPDAF309:~/prac7$ ./memory
```

```
Code address      : 0x5bc1ac2101a9
```

```
Global address    : 0x5bc1ac213010
```

```
Static address    : 0x5bc1ac213014
```

```
Heap address      : 0x5bc1ceaad010
```

```
Stack address     : 0x7ffd6137a85c
```

```
ramya@LAPTOP-OPDAF309:~/prac7$ ./memory
```

```
Code address      : 0x5918649d61a9
```

```
Global address    : 0x5918649d9010
```

```
Static address    : 0x5918649d9014
```

```
Heap address      : 0x5918868b1010
```

```
Stack address     : 0x7fffffb183b7c
```

```
ramya@LAPTOP-OPDAF309:~/prac7$ |
```

TASK 2: Make use of the /proc//maps file and memory analysis tools to study the memory organization of a running process. Prepare a report explaining virtual memory mappings. give the commands for this in one terminal only



```
GNU nano 8.7.1 memory.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int global_var = 10;

void code_function()
{
}

int main()
{
    static int static_var = 20;
    int stack_var = 30;
    int *heap_var = malloc(sizeof(int));

    *heap_var = 40;

    printf("PID          : %d\n", getpid());
    printf("Code address   : %p\n", (void *)code_function);
    printf("Global address  : %p\n", (void *)&global_var);
    printf("Static address  : %p\n", (void *)&static_var);
    printf("Heap address    : %p\n", (void *)heap_var);
    printf("Stack address   : %p\n", (void *)&stack_var);

    printf("\nProcess is running... Press Ctrl+C to stop.\n");

    while (1)
    {
        sleep(1);
    }

    free(heap_var);
    return 0;
}
```

```
ramya@LAPTOP-OPDAF309: ~ - x + v
ramya@LAPTOP-OPDAF309:~/prac7$ nano memory.c
ramya@LAPTOP-OPDAF309:~/prac7$ gcc memory.c -o memory
ramya@LAPTOP-OPDAF309:~/prac7$ ./memory
PID          : 1263
Code address  : 0x5e70389211c9
Global address : 0x5e7038924010
Static address : 0x5e7038924014
Heap address   : 0x5e706eaf6010
Stack address  : 0x7fff8e45c9ec

Process is running... Press Ctrl+C to stop.
^Z
[1]+  Stopped                  ./memory
ramya@LAPTOP-OPDAF309:~/prac7$ PID=$(pgrep -n memory)
echo $PID
1263
ramya@LAPTOP-OPDAF309:~/prac7$ cat /proc/$PID/maps
5e7038920000-5e7038921000 r--p 00000000 08:30 13254
me/ramya/prac7/memory
5e7038921000-5e7038922000 r-xp 00001000 08:30 13254
me/ramya/prac7/memory
5e7038922000-5e7038923000 r--p 00002000 08:30 13254
me/ramya/prac7/memory
5e7038923000-5e7038924000 r--p 00002000 08:30 13254
me/ramya/prac7/memory
5e7038924000-5e7038925000 rw-p 00003000 08:30 13254
me/ramya/prac7/memory
5e706eaf6000-5e706eb17000 rw-p 00000000 00:00 0
ap]
7eb110200000-7eb110228000 r--p 00000000 08:30 13647
r/lib/x86_64-linux-gnu/libc.so.6
7eb110228000-7eb1103c0000 r-xp 00028000 08:30 13647
r/lib/x86_64-linux-gnu/libc.so.6
7eb1103c0000-7eb11040e000 r--p 001c0000 08:30 13647
r/lib/x86_64-linux-gnu/libc.so.6
7eb11040e000-7eb110412000 r--p 0020d000 08:30 13647
r/lib/x86_64-linux-gnu/libc.so.6
7eb110412000-7eb110414000 rw-p 00211000 08:30 13647
r/lib/x86_64-linux-gnu/libc.so.6
7eb110414000-7eb110421000 rw-p 00000000 00:00 0
7eb110537000-7eb11053a000 rw-p 00000000 00:00 0
```

```

7fff8e43e000-7fff8e45f000 rw-p 00000000 00:00 0
ack]
ramya@LAPTOP-OPDAF309:~/prac7$ cat /proc/$PID/maps | grep -E '\[heap\]|memory'
5e7038920000-5e7038921000 r--p 00000000 08:30 13254
me/ramya/prac7/memory
5e7038921000-5e7038922000 r-xp 00001000 08:30 13254
me/ramya/prac7/memory
5e7038922000-5e7038923000 r--p 00002000 08:30 13254
me/ramya/prac7/memory
5e7038923000-5e7038924000 r--p 00002000 08:30 13254
me/ramya/prac7/memory
5e7038924000-5e7038925000 rw-p 00003000 08:30 13254
me/ramya/prac7/memory
5e706eaf6000-5e706eb17000 rw-p 00000000 00:00 0
ap]
7fff8e43e000-7fff8e45f000 rw-p 00000000 00:00 0
ack]
ramya@LAPTOP-OPDAF309:~/prac7$ cat /proc/$PID/status | grep -E 'VmP
e|VmRSS|VmData|VmStk|VmExe|VmLib'
VmPeak:      2768 kB
VmSize:      2768 kB
VmRSS:       1880 kB
VmData:      224 kB
VmStk:       132 kB
VmExe:        4 kB
VmLib:      1828 kB
ramya@LAPTOP-OPDAF309:~/prac7$ pmap $PID
1263:      ./memory
00005e7038920000      4K r---- memory
00005e7038921000      4K r-x-- memory
00005e7038922000      4K r---- memory
00005e7038923000      4K r---- memory
00005e7038924000      4K rw--- memory
00005e706eaf6000    132K rw--- [ anon ]
00007eb110200000    160K r---- libc.so.6
00007eb110228000   1632K r-x-- libc.so.6
00007eb1103c0000    312K r---- libc.so.6
00007eb11040e000     16K r---- libc.so.6
00007eb110412000      8K rw--- libc.so.6
00007eb110414000     52K rw--- [ anon ]

```

```
00007eb110537000      12K rw--- [ anon ]
00007eb110540000       8K rw--- [ anon ]
00007eb110542000      16K r---- [ anon ]
00007eb110546000       8K r---- [ anon ]
00007eb110548000       8K r-x-- [ anon ]
00007eb11054a000       4K r---- ld-linux-x86-64.so.2
00007eb11054b000     188K r-x-- ld-linux-x86-64.so.2
00007eb11057a000     44K r---- ld-linux-x86-64.so.2
00007eb110585000       8K r---- ld-linux-x86-64.so.2
00007eb110587000       4K rw--- ld-linux-x86-64.so.2
00007eb110588000       4K rw--- [ anon ]
00007fff8e43e000    132K rw--- [ stack ]
total                2768K
```

```
ramya@LAPTOP-OPDAF309:~/prac7$ fg
```

```
./memory
```

```
^C
```